calculate_postTARE_ts_aws_peak_demand_PA_preliminary

April 16, 2026

Post-TARE Model Run: Timeseries Peak Load Analysis (AWS / BuildStockQuery)

Author: Jordan M. Joseph, PhD — Carnegie Mellon University

Queries the AWS-hosted ResStock EUSS 2022.1.1 timeseries database via BuildStockQuery to compute **county-level peak load changes** under two adoption scenarios: - **(a) 100% adoption counterfactual** — all filtered building IDs adopt (upper bound) - **(b) Economically-constrained adoption** — only Tier 1 + Tier 2 adopters (TARE output)

Prerequisite: Steps 0–0c must be run first (TARE data loaded, MP selected).

Dataset scope: ResStock 2022.1.1 (EUSS, AMY2018). ResStock 2025.1 is future work.

Primary test case: Allegheny County, PA (FIPS 42003) — validate here before scaling.

See methodology notes in `peak_load_methodology.md`.

Step 0: Imports and Configuration

[1]:
```
import os
import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

from config import PROJECT_ROOT
from cmu_tare_model.constants import (
    ALLOWED_HOUSING_TYPES,
    VALID_MENU_MPS,
    VERBOSE,
    REMDB_COST_SCENARIO_KEYS,
    RCM_MODELS,
    PRIVATE_DISCOUNT_RATE_SHORT_KEYS,
)

from cmu_tare_model.utils.column_names import (
    create_npv_col,
    create_capital_col,
```

```
)

from cmu_tare_model.utils.load_exported_results_to_df import␣
 ↪load_measure_package_data

from cmu_tare_model.adoption_kpis.kpi_functions import (
    mp_to_upgrade,
    load_euss_baseline,
    load_euss_upgrade,
    calculate_price_ratios,
    compute_thermal_cop_by_state,
    compute_spark_gap_metrics,
    compute_scenario_demand,
    aggregate_demand_by_state,
    FUEL_PRICES_PATH,
    SHAPEFILE_PATH,
    HEATING_FUEL_COLS,
    HP_BACKUP_ELEC_COL,
    HP_FANS_PUMPS_COL,
)
from cmu_tare_model.adoption_kpis.visualize_geospatial_data import (
    prepare_state_geodataframe,
    create_choropleth_map,
)

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 60)

print("  Imports loaded")
```

```
Project root directory: c:\users\jorda\desktop\projects\cmu-tare-model
  Imports loaded
```

Step 0b: Measure Package Selection

[2]:
```
SELECTABLE_MPS = [mp for mp in VALID_MENU_MPS if mp != 0]

try:
    _ = input_measure_package
    batch_mode = True
    selected_mps = [int(input_measure_package)]
    print(f"BATCH MODE: Running for MP{selected_mps[0]}")
except NameError:
    batch_mode = False
    print(f"Available measure packages: {SELECTABLE_MPS}")
    mp_input = input("Enter MP numbers (comma-separated, or 'all'): ").strip()
    if mp_input.lower() == 'all':
```

```
        selected_mps = SELECTABLE_MPS
    else:
        selected_mps = [int(x.strip()) for x in mp_input.split(',') if x.
 ↪strip().isdigit()]
        selected_mps = [mp for mp in selected_mps if mp in SELECTABLE_MPS]
    if not selected_mps:
        selected_mps = [4]
        print("No valid MPs selected. Defaulting to MP4.")

print(f"\nSelected measure packages: {selected_mps}")
```

```
Available measure packages: [3, 4]

Selected measure packages: [3]
```

Step 0c: Load TARE Model Data (Measure Packages 3, 4)

Load pre-computed TARE model outputs for the selected measure packages. Required for Step 4d (Private NPV extraction).

If the top-section data loading cells have already been run, this will reuse `DATAFRAMES_BY_MP`.

[3]:
```
# ============================================================================
# STEP 0c: LOAD TARE MODEL DATA (for NPV extraction)
# ============================================================================
# Check if DATAFRAMES_BY_MP was already loaded by the top-section cells.
# If not, prompt for the output folder and load for selected MPs.

try:
    _ = DATAFRAMES_BY_MP
    print(f"DATAFRAMES_BY_MP already loaded: {list(DATAFRAMES_BY_MP.keys())}")
except NameError:
    print("DATAFRAMES_BY_MP not found - loading TARE model outputs...")

    # Check if output_folder_path is already defined (from top section)
    try:
        _ = output_folder_path
        print(f"  Using existing output_folder_path: {output_folder_path}")
    except NameError:
        output_folder_path = os.path.join(PROJECT_ROOT, "cmu_tare_model",␣
 ↪"output_results")
        location_id = input("Enter location ID (e.g., 'National' or 'PA'): ").
 ↪strip()
        model_run_date_time = input("Enter model run timestamp␣
 ↪(YYYY-MM-DD_HH-MM): ").strip()
        print(f"  output_folder_path: {output_folder_path}")
        print(f"  location_id: {location_id}")
```

```
        print(f"  model_run_date_time: {model_run_date_time}")

    DATAFRAMES_BY_MP = {}
    for mp in selected_mps:
        DATAFRAMES_BY_MP[mp] = load_measure_package_data(
            mp, output_folder_path, location_id, model_run_date_time
        )

    print(f"\n  Loaded TARE data for MPs: {list(DATAFRAMES_BY_MP.keys())}")
```

```
DATAFRAMES_BY_MP not found - loading TARE model outputs…
  output_folder_path: c:\users\jorda\desktop\projects\cmu-tare-
model\cmu_tare_model\output_results
  location_id: National
  model_run_date_time: 2026-04-10_00-05
Loading MP3 data…
  fixed_base:
MP3 loading complete!


  Loaded TARE data for MPs: [3]
```

Step 1: BuildStockQuery Installation and AWS Configuration

Installs BuildStockQuery (NREL SWR-23-58) and verifies AWS credentials are configured. BuildStockQuery connects to the OEDI data lake via AWS Athena and returns results as pandas DataFrames.

0.0.1 One-time setup (do once per machine)

1. Install packages (in the `cmu-tare-model` conda environment):

```
conda activate cmu-tare-model
pip install boto3
pip install git+https://github.com/NREL/buildstock-query.git
```

2. Create an AWS IAM Access Key: 1. Sign in to the AWS Console 2. Go to **IAM → Users → (your user) → Security credentials** 3. Click **Create access key** 4. Select use case: **Command Line Interface (CLI)** 5. Check the confirmation box and click **Next → Create access key** 6. **Copy both values immediately** — the Secret Access Key is only shown once: - Access Key ID (20 chars, starts with `AKIA`) - Secret Access Key (40-char random string)

3. Configure the AWS CLI:

```
aws configure
```

Enter the following when prompted: | Prompt | Value | |———|——-| | AWS Access Key ID | `AKIA...` (from step 2) | | AWS Secret Access Key | (40-char string from step 2) | | Default region name | `us-west-2` | | Default output format | `json` |

4. Verify credentials work:

`aws sts get-caller-identity`

You should see your Account ID and ARN printed — no errors.

0.0.2 Required IAM permissions

The IAM user needs these managed policies (or equivalent): - `AmazonAthenaFullAccess` — query the OEDI data lake - `AmazonS3ReadOnlyAccess` — read Athena query results from S3

0.0.3 References

- BuildStockQuery docs: https://github.com/NREL/buildstock-query/wiki
- BuildStockQuery examples: https://github.com/NREL/buildstock-query/tree/main/example_usage
- OEDI S3 browser: https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2F
- AWS CLI install: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

[4]:
```
# CONTEXT: Setting up BuildStockQuery for ResStock EUSS 2022.1.1 timeseries␣
 ↪queries
# via AWS Athena on the OEDI data lake. Python 3.11, conda env: cmu-tare-model.
#
# TASK:
#   1. Check if buildstock_query is importable; if not, print install␣
 ↪instructions.
#   2. Check AWS credentials are configured (boto3 STS get_caller_identity).
#   3. Print the confirmed AWS region and identity as a sanity check.
#   4. Import ResStockQuery (or equivalent BuildStockQuery entry point).
#
# INPUTS: None (environment check only)
# OUTPUTS: Printed confirmation. `bsq_available` (bool). `ResStockQuery` class␣
 ↪imported.
# CONSTRAINTS: Type hints on any helper functions. Fail fast with a clear error␣
 ↪message
#              if AWS credentials are missing. Do not hardcode any credentials.

#   1. Check BuildStockQuery availability
bsq_available: bool = False
BuildStockQuery = None

try:
    from buildstock_query import BuildStockQuery  # type: ignore[import-untyped]
    bsq_available = True
    print(f" buildstock_query imported (BuildStockQuery class:␣
 ↪{BuildStockQuery})")
except ImportError:
```

```
    print(
        "  buildstock_query is NOT installed.\n"
        "   Install with:\n"
        "     pip install git+https://github.com/NREL/buildstock-query\n"
        "   Or for full features:\n"
        "     pip install git+https://github.com/NREL/
 ↪buildstock-query#egg=buildstock-query[full]"
    )

#   2. Check AWS credentials
aws_identity: dict | None = None
aws_region: str | None = None

if bsq_available:
    import boto3
    from botocore.exceptions import (
        NoCredentialsError,
        ClientError,
        TokenRetrievalError,
    )

    session = boto3.session.Session()
    aws_region = session.region_name

    try:
        sts = session.client("sts")
        aws_identity = sts.get_caller_identity()
        print(f"  AWS credentials valid")
        print(f"   Account : {aws_identity['Account']}")
        print(f"   ARN     : {aws_identity['Arn']}")
        print(f"   Region  : {aws_region}")
    except NoCredentialsError:
        raise RuntimeError(
            "AWS credentials not found. Run `aws configure` or set "
            "AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY environment variables."
        )
    except TokenRetrievalError as e:
        raise RuntimeError(
            f"AWS SSO token retrieval failed - run `aws sso login` first.\n ␣
 ↪{e}"
        )
    except ClientError as e:
        raise RuntimeError(f"AWS STS call failed: {e}")
else:
    print("  Skipping AWS credential check (buildstock_query not installed).")

print("\n  Step 1 COMPLETE")
```

```
INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials
```

```
 buildstock_query imported (BuildStockQuery class: <class
'buildstock_query.main.BuildStockQuery'>)
 AWS credentials valid
  Account : 069765036196
  ARN     : arn:aws:iam::069765036196:user/jordanjo@alumni.cmu.edu
  Region  : us-west-2

 Step 1 COMPLETE
```

Step 2: OEDI Table Discovery — Confirm EUSS 2022.1.1 Schema

Before writing any queries, we need to confirm the exact Athena database name, table names, and column schema for ResStock EUSS 2022.1.1 timeseries data.

Key unknowns to resolve in this step: - What is the Athena database name for EUSS 2022.1.1? - What are the timeseries table names (baseline = MP0, upgrade = MP3/MP4)? - What county-level geography columns exist? (FIPS code? GISJOIN? county name string?) - What is the exact column name and format for `bldg_id` in the timeseries table? - What are the electricity end-use column names for total consumption per hour? - What is the `timestamp` column name and format (ISO string? epoch integer? hour index 1–8760?)

OEDI S3 path (confirm via browser): `s3://oedi-data-lake/nrel-pds-building-stock/end-use-load-pr`

Expected output of this step: A printed schema summary that confirms all column names needed for Steps 3–6.

[5]:
```
# Imports

import boto3
from typing import Any

AWS_REGION: str = "us-west-2"

s3 = boto3.client("s3", region_name=AWS_REGION)
athena = boto3.client("athena", region_name=AWS_REGION)
glue = boto3.client("glue", region_name=AWS_REGION)
```

```
INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials
```

[6]:
```
# Check that boto3 can access AWS and list S3 buckets (sanity check for␣
 ↪credentials and region)
BUCKET_NAME = input("Enter the bucket name (e.g.,␣
 ↪resstock-euss-query-results-{your-account-id}): ")
print(f"Bucket name is set to: {BUCKET_NAME}")
```

```
buckets = [bucket["Name"] for bucket in s3.list_buckets()["Buckets"]]
print("  Bucket found" if BUCKET_NAME in buckets else "  Bucket not found -␣
 ↪check S3 console")
```

```
Bucket name is set to: resstock-euss-query-results-069765036196
  Bucket found
```

[7]:
```
# Check that the specified Athena workgroup exists (sanity check for Athena␣
 ↪access and correct workgroup name)
WORKGROUP_NAME = input("Enter the Athena workgroup name (e.g., resstock-euss):␣
 ↪")
print(f"Workgroup name is set to: {WORKGROUP_NAME}")

workgroup_names = [wg["Name"] for wg in athena.list_work_groups()["WorkGroups"]]
print("  Workgroup found" if WORKGROUP_NAME in workgroup_names else "  Workgroup␣
 ↪not found")
```

```
Workgroup name is set to: resstock-euss
  Workgroup found
```

[8]:
```
# Check that the specified Glue database exists (sanity check for Glue access␣
 ↪and correct database name)
DB_NAME = input("Enter the Glue database name (e.g., euss-oedi): ")
print(f"Glue database name is set to: {DB_NAME}")

db_names = [db["Name"] for db in glue.get_databases()["DatabaseList"]]
print("  Database found" if DB_NAME in db_names else "  Database not found")
```

```
Glue database name is set to: euss-oedi
  Database found
```

[9]:
```
# Expected: two tables - timeseries and metadata
# Note the exact names - needed for the rename step in Part 5
tables = glue.get_tables(DatabaseName=DB_NAME)["TableList"]
table_names = [t["Name"] for t in tables]
print(f"Tables in {DB_NAME}: {table_names}")
```

```
Tables in euss-oedi: ['resstock_amy2018_release_1_1_by_state',
'resstock_amy2018_release_1_1_metadata']
```

[10]:
```
# Fix the timeseries table
TS_TABLE = input(f"Enter the name of the table to fix (e.g.,␣
 ↪resstock_amy2018_release_1_1_by_state): ").strip()

# Step 1: Fetch the full table definition before deleting
table = glue.get_table(DatabaseName=DB_NAME, Name=TS_TABLE)["Table"]

# Step 2: Delete the crawler-generated table
glue.delete_table(DatabaseName=DB_NAME, Name=TS_TABLE)
```

```
print(f"  Deleted: {TS_TABLE}")

# Step 3: Fix the upgrade partition key type in our local copy
for key in table["PartitionKeys"]:
    if key["Name"] == "upgrade":
        key["Type"] = "int"
        print(f"  Fixed: upgrade → int")

# Step 4: Recreate with corrected schema
glue.create_table(
    DatabaseName=DB_NAME,
    TableInput={
        "Name": table["Name"],
        "StorageDescriptor": table["StorageDescriptor"],
        "PartitionKeys": table["PartitionKeys"],
        "TableType": table.get("TableType", ""),
        "Parameters": table.get("Parameters", {}),
    }
)
print(f"  Recreated: {TS_TABLE} with upgrade as int")

# Step 5: Verify
updated = glue.get_table(DatabaseName=DB_NAME, Name=TS_TABLE)["Table"]
for key in updated["PartitionKeys"]:
    if key["Name"] == "upgrade":
        print(f"  Confirmed upgrade type: {key['Type']}")
```

```
 Deleted: resstock_amy2018_release_1_1_by_state
  Fixed: upgrade → int
 Recreated: resstock_amy2018_release_1_1_by_state with upgrade as int
 Confirmed upgrade type: int
```

[11]:
```
#␣
 ↪============================================================================
#  Configuration Check
#␣
 ↪============================================================================

print(f"""
============================================================
CONFIGURATION CHECK:
============================================================
AWS_REGION: {AWS_REGION}
BUCKET_NAME: {BUCKET_NAME}
WORKGROUP_NAME: {WORKGROUP_NAME}

DB_NAME: {DB_NAME}
```

```
TS_TABLE (Table that was fixed): {TS_TABLE}
""")
```

```
============================================================
CONFIGURATION CHECK:
============================================================
AWS_REGION: us-west-2
BUCKET_NAME: resstock-euss-query-results-069765036196
WORKGROUP_NAME: resstock-euss

DB_NAME: euss-oedi
TS_TABLE (Table that was fixed): resstock_amy2018_release_1_1_by_state
```

[12]:
```
#␣
 ↪================================================================================
# 1. Read table definition to get S3 location and partition key order
#␣
 ↪================================================================================

table_def: dict[str, Any] = glue.get_table(
    DatabaseName=DB_NAME, Name=TS_TABLE
)["Table"]

s3_location: str = table_def["StorageDescriptor"]["Location"]
partition_keys: list[str] = [pk["Name"] for pk in table_def["PartitionKeys"]]
storage_descriptor: dict[str, Any] = table_def["StorageDescriptor"]

print(f"  Table S3 location : {s3_location}")
print(f"  Partition keys    : {partition_keys}")

assert "by_state" in s3_location, (
    f"Unexpected S3 location: {s3_location}\n"
    f"Expected a path containing 'by_state' - check the Glue table definition."
)

# ========== Parse bucket and prefix from the table's S3 location ==========
# s3_location format: "s3://bucket-name/path/to/prefix/"
# We parse both from this single source of truth.
_s3_no_scheme: str = s3_location.removeprefix("s3://")
_slash_idx: int = _s3_no_scheme.index("/")
DATA_BUCKET: str = _s3_no_scheme[:_slash_idx]          # e.g. "oedi-data-lake"
s3_prefix: str = _s3_no_scheme[_slash_idx + 1:]         # everything after␣
 ↪bucket/

if not s3_prefix.endswith("/"):
```

```
    s3_prefix += "/"

print(f"  Parsed S3 bucket  : {DATA_BUCKET}")
print(f"  Parsed S3 prefix  : {s3_prefix}")
```

```
  Table S3 location : s3://oedi-data-lake/nrel-pds-building-stock/end-use-load-
profiles-for-us-building-stock/2022/resstock_amy2018_release_1.1/timeseries_indi
vidual_buildings/by_state/
  Partition keys    : ['upgrade', 'state']
  Parsed S3 bucket  : oedi-data-lake
  Parsed S3 prefix  : nrel-pds-building-stock/end-use-load-profiles-for-us-
building-stock/2022/resstock_amy2018_release_1.1/timeseries_individual_buildings
/by_state/
```

[13]:
```
#␣
 ↪==============================================================================
# 2. Discover partition paths from S3
#␣
 ↪==============================================================================

print(f"\n  Listing S3 prefixes under: s3://{DATA_BUCKET}/{s3_prefix}")
print(f"  (This reads only prefix metadata from the public OEDI bucket - no␣
 ↪data scanned)")

# List upgrade=N/ prefixes (depth 1)
upgrade_prefixes: list[str] = []
paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(
    Bucket=DATA_BUCKET, Prefix=s3_prefix, Delimiter="/"
):
    for cp in page.get("CommonPrefixes", []):
        upgrade_prefixes.append(cp["Prefix"])

print(f"  Found {len(upgrade_prefixes)} upgrade-level prefixes")

# List upgrade=N/state=XX/ prefixes (depth 2)
partition_paths: list[tuple[str, str]] = []

for up_prefix in upgrade_prefixes:
    for page in paginator.paginate(
        Bucket=DATA_BUCKET, Prefix=up_prefix, Delimiter="/"
    ):
        for cp in page.get("CommonPrefixes", []):
            parts = cp["Prefix"].rstrip("/").split("/")
            upgrade_part = [p for p in parts if p.startswith("upgrade=")]
            state_part = [p for p in parts if p.startswith("state=")]
            if upgrade_part and state_part:
```

```
                upgrade_val = upgrade_part[0].split("=")[1]
                state_val = state_part[0].split("=")[1]
                partition_paths.append((upgrade_val, state_val))

print(f"  Found {len(partition_paths)} partition paths (expected: 539)")
```

```
  Listing S3 prefixes under: s3://oedi-data-lake/nrel-pds-building-stock/end-
use-load-profiles-for-us-building-stock/2022/resstock_amy2018_release_1.1/timese
ries_individual_buildings/by_state/
  (This reads only prefix metadata from the public OEDI bucket - no data
scanned)
  Found 11 upgrade-level prefixes
  Found 539 partition paths (expected: 539)
```

[14]:
```
#␣
 ↪==========================================================================
# 3. Build partition entries and register in batches of 100
#␣
 ↪==========================================================================
# CRITICAL: The Values list must match the ORDER of table_def["PartitionKeys"].
# The crawler may have detected them as [upgrade, state] or [state, upgrade].
# We read the order from the table definition and map accordingly.

def build_partition_value_list(
    upgrade_val: str, state_val: str, key_order: list[str]
) -> list[str]:
    """Map partition values to the correct order for the Glue table.

    Args:
        upgrade_val: The upgrade partition value (e.g., "3").
        state_val: The state partition value (e.g., "PA").
        key_order: Partition key names in the order defined by the table.

    Returns:
        Values list matching the table's partition key order.

    Raises:
        ValueError: If key_order contains unexpected partition key names.
    """
    mapping = {"upgrade": upgrade_val, "state": state_val}
    try:
        return [mapping[k] for k in key_order]
    except KeyError as e:
        raise ValueError(
            f"Unexpected partition key {e} in table definition.\n"
            f"  Expected keys: 'upgrade', 'state'\n"
```

```
            f"  Table defines: {key_order}"
        ) from e


def build_partition_input(
    upgrade_val: str,
    state_val: str,
    key_order: list[str],
    base_sd: dict[str, Any],
    base_s3: str,
    bucket: str = DATA_BUCKET,
) -> dict[str, Any]:
    """Build a single PartitionInput dict for batch_create_partition.

    Args:
        upgrade_val: Upgrade number as string.
        state_val: State abbreviation.
        key_order: Partition key order from table definition.
        base_sd: StorageDescriptor from the table (template for partitions).
        base_s3: Base S3 prefix (without partition directories).
        bucket: S3 bucket name.

    Returns:
        Dict suitable for Glue batch_create_partition PartitionInputList.
    """
    values = build_partition_value_list(upgrade_val, state_val, key_order)
    partition_s3 = f"s3://{bucket}/{base_s3}upgrade={upgrade_val}/
 ↪state={state_val}/"

    # Each partition's StorageDescriptor is the same as the table's,
    # except with a Location pointing to this specific partition's S3 path.
    partition_sd = {
        "Columns": base_sd["Columns"],
        "InputFormat": base_sd["InputFormat"],
        "OutputFormat": base_sd["OutputFormat"],
        "SerdeInfo": base_sd["SerdeInfo"],
        "Location": partition_s3,
    }
    return {"Values": values, "StorageDescriptor": partition_sd}


# Build all partition inputs
partition_inputs: list[dict[str, Any]] = [
    build_partition_input(up, st, partition_keys, storage_descriptor,␣
 ↪s3_prefix, DATA_BUCKET)
    for up, st in partition_paths
]
```

```
# Register in batches of 100 (API limit)
BATCH_SIZE: int = 100
registered: int = 0
already_exists: int = 0
errors: list[dict] = []

print(f"\n  Registering {len(partition_inputs)} partitions in batches of␣
 ↪{BATCH_SIZE}...")

for i in range(0, len(partition_inputs), BATCH_SIZE):
    batch = partition_inputs[i : i + BATCH_SIZE]
    response = glue.batch_create_partition(
        DatabaseName=DB_NAME,
        TableName=TS_TABLE,
        PartitionInputList=batch,
    )
    # Count successes and failures
    batch_errors = response.get("Errors", [])
    for err in batch_errors:
        if err["ErrorDetail"]["ErrorCode"] == "AlreadyExistsException":
            already_exists += 1
        else:
            errors.append(err)
    registered += len(batch) - len(batch_errors) + already_exists

    # Progress indicator every 200
    total_processed = min(i + BATCH_SIZE, len(partition_inputs))
    if total_processed % 200 == 0 or total_processed == len(partition_inputs):
        print(f"    Processed {total_processed} / {len(partition_inputs)}")
```

```
  Registering 539 partitions in batches of 100…
    Processed 200 / 539
    Processed 400 / 539
    Processed 539 / 539
```

[15]:
```
#␣
 ↪============================================================================
# 4. Summary
#␣
 ↪============================================================================

new_registrations = len(partition_inputs) - already_exists - len(errors)
print(f"\n  Partition registration complete:")
print(f"    New registrations : {new_registrations}")
print(f"    Already existed   : {already_exists}")
```

```
print(f"    Errors            : {len(errors)}")
if errors:
    print(f"      Error details:")
    for err in errors[:5]:  # Show first 5
        print(f"      {err['PartitionValues']}: {err['ErrorDetail']}")
```

```
 Partition registration complete:
   New registrations : 539
   Already existed   : 0
   Errors            : 0
```

[16]:
```
#␣
 ↪============================================================================
# 5. Verify total partition count
#␣
 ↪============================================================================

partitions_response = glue.get_partitions(
    DatabaseName=DB_NAME, TableName=TS_TABLE, MaxResults=1
)
# get_partitions is paginated; use a count query instead
segment_count: int = 0
next_token: str | None = None

while True:
    kwargs: dict[str, Any] = {
        "DatabaseName": DB_NAME,
        "TableName": TS_TABLE,
    }
    if next_token:
        kwargs["NextToken"] = next_token
    resp = glue.get_partitions(**kwargs)
    segment_count += len(resp["Partitions"])
    next_token = resp.get("NextToken")
    if not next_token:
        break

print(f"\n  Total partitions in catalog: {segment_count}")
if segment_count == 539:
    print(f"    MATCHES expected value (539 = 49 states × 11 upgrades)")
else:
    print(f"    MISMATCH: expected 539, found {segment_count}")
    print(f"     Investigate before proceeding to verification queries.")

print("\n  Partition registration complete")
```

```
Total partitions in catalog: 539
  MATCHES expected value (539 = 49 states × 11 upgrades)

Partition registration complete
```

[ ]:

[17]:
```python
# ============================================================================
# PRE-FLIGHT P0: Verify S3 Write Access to Athena Results Bucket
# ============================================================================
# CONTEXT: Athena writes query results to S3 before returning them. We need
# write access to the results bucket. The plan calls for applying an inline
# IAM policy, but this user lacks iam:PutUserPolicy. Instead, verify that
# write access already works (the policy may have been attached via console).
#
# TEST: Write a small test object, read it back, then delete it.

import json
import boto3

ATHENA_RESULTS_S3: str = f"s3://{BUCKET_NAME}/query-results/"
TEST_KEY: str = "query-results/_preflight_write_test.txt"

caller = sts.get_caller_identity()
user_arn: str = caller["Arn"]
print(f"IAM identity: {user_arn}")
print(f"Results bucket: s3://{BUCKET_NAME}/")

# Test 1: Write a small object
try:
    s3.put_object(
        Bucket=BUCKET_NAME,
        Key=TEST_KEY,
        Body=b"preflight write test",
    )
    print(f"  S3 PutObject succeeded (key: {TEST_KEY})")
except Exception as e:
    raise RuntimeError(
        f"S3 write test FAILED on bucket '{BUCKET_NAME}'.\n"
        f"  Error: {e}\n"
        f"  Action needed: attach the AthenaResultsBucketWrite policy to user "
        f"'{user_arn}' via the AWS IAM console (this user lacks iam:
 ↪PutUserPolicy "
        f"to self-apply it)."
    ) from e

# Test 2: Read it back
```

```
try:
    obj = s3.get_object(Bucket=BUCKET_NAME, Key=TEST_KEY)
    body = obj["Body"].read().decode()
    assert body == "preflight write test", f"Read-back mismatch: {body!r}"
    print(f"  S3 GetObject succeeded - content verified")
except Exception as e:
    raise RuntimeError(f"S3 read-back test FAILED: {e}") from e

# Test 3: Clean up
try:
    s3.delete_object(Bucket=BUCKET_NAME, Key=TEST_KEY)
    print(f"  S3 DeleteObject succeeded - test object removed")
except Exception as e:
    print(f"  S3 DeleteObject failed (non-critical): {e}")

# Test 4: ListBucket
try:
    s3.list_objects_v2(Bucket=BUCKET_NAME, Prefix="query-results/", MaxKeys=1)
    print(f"  S3 ListBucket succeeded")
except Exception as e:
    raise RuntimeError(f"S3 ListBucket test FAILED: {e}") from e

print(f"\n  Pre-flight P0 validated: full read/write/delete/list access to s3://
  ↪{BUCKET_NAME}/")
```

```
IAM identity: arn:aws:iam::069765036196:user/jordanjo@alumni.cmu.edu
Results bucket: s3://resstock-euss-query-results-069765036196/
  S3 PutObject succeeded (key: query-results/_preflight_write_test.txt)
  S3 GetObject succeeded - content verified
  S3 DeleteObject succeeded - test object removed
  S3 ListBucket succeeded

  Pre-flight P0 validated: full read/write/delete/list access to s3://resstock-
euss-query-results-069765036196/
```

[18]:
```
# ============================================================================
# PRE-FLIGHT P1: Workgroup Diagnostic
# ============================================================================
# CONTEXT: Diagnose Athena workgroup configuration before running queries.
# TASK: Print workgroup output location, engine version, and enforce flag.

wg_info = athena.get_work_group(WorkGroup=WORKGROUP_NAME)["WorkGroup"]
config = wg_info["Configuration"]
result_config = config.get("ResultConfiguration", {})
engine = config.get("EngineVersion", {})

print(f"Workgroup         : {wg_info['Name']}")
```

```
print(f"Output location   : {result_config.get('OutputLocation', '(not set)')}")
print(f"Engine version    : {engine.get('SelectedEngineVersion', '(default)')}")
print(f"Effective engine  : {engine.get('EffectiveEngineVersion',␣
 ↪'(unknown)')}")
print(f"Enforce config    : {config.get('EnforceWorkGroupConfiguration',␣
 ↪False)}")

expected_out = f"s3://{BUCKET_NAME}/query-results/"
actual_out = result_config.get("OutputLocation", "")
if not actual_out.startswith(f"s3://{BUCKET_NAME}"):
    print(f"\n  Workgroup output bucket does NOT match results bucket.")
    print(f"  Expected: {expected_out}")
    print(f"  Actual  : {actual_out}")
    print(f"  Either update the workgroup or pass output_location explicitly")
    print(f"  to run_athena_query() on every call.")
else:
    print(f"\n  Output location matches results bucket")

print(f"\n  Pre-flight P1 validated")
```

```
Workgroup         : resstock-euss
Output location   : s3://resstock-euss-query-results-069765036196/
Engine version    : AUTO
Effective engine  : Athena engine version 3
Enforce config    : True

  Output location matches results bucket

  Pre-flight P1 validated
```

[19]:
```
# ============================================================================
# ATHENA VERIFICATION QUERIES
# ============================================================================
# Confirms the Glue table + partition registration actually work end-to-end
# by running two reference queries with known expected values.
#
# Query 1: Row count for PA baseline (validates partition pruning)
#          Expected: 807,742,080 rows, ~0 MB scanned
# Query 2: Unique buildings in Allegheny County, PA, baseline
#          Expected: 2,434 buildings
#
# WHY boto3 instead of the Athena console: reproducibility for the paper's
# supplementary methods. Every query run here is code that can be committed
# and re-run by a reviewer.
# ============================================================================

import time
```

```python
from typing import Any

#   Configuration
ATHENA_RESULTS_S3: str = f"s3://{BUCKET_NAME}/query-results/"

#   Step 1: Discover timeseries column schema from Glue (no data scanned)
# WHY: Before writing any SELECT, we need to know the actual column names.
# Glue's get_table() returns the schema for free - no Athena query needed.
table_def = glue.get_table(DatabaseName=DB_NAME, Name=TS_TABLE)["Table"]
columns: list[dict[str, str]] = table_def["StorageDescriptor"]["Columns"]

print(f"  Timeseries table schema ({len(columns)} columns)  ")
# Print likely-relevant columns first: anything that looks like id/time/county/
 ↪electricity
for col in columns:
    name = col["Name"]
    dtype = col["Type"]
    cl = name.lower()
    if any(tok in cl for tok in ("bldg", "building", "time", "county", "state",␣
 ↪"electricity")):
        print(f"  {name:<60s} {dtype}")

print(f"\n  (Full column list available in `columns` variable - {len(columns)}␣
 ↪total)")

#   Step 2: Helper function for running Athena queries
def run_athena_query(
    query: str,
    workgroup: str = WORKGROUP_NAME,
    output_location: str = ATHENA_RESULTS_S3,
    poll_interval_s: float = 1.0,
    timeout_s: float = 120.0,
) -> dict[str, Any]:
    """Run an Athena query synchronously and return results + statistics.

    Starts an async Athena query execution, polls until completion, then
    fetches the result rows and scan statistics. Synchronous wrapper
    chosen for notebook ergonomics - true async isn't needed here.

    Args:
        query: SQL string. Quote hyphenated database names: "euss-oedi".
        workgroup: Athena workgroup name.
        output_location: S3 URI where Athena writes result files.
        poll_interval_s: Seconds between status polls.
        timeout_s: Maximum seconds to wait before raising TimeoutError.

    Returns:
```

```
        Dict with keys:
          - 'rows'     : list[list[str]], result rows (first row is headers)
          - 'scanned_bytes' : int, bytes scanned (0 = partition metadata only)
          - 'runtime_ms'    : int, total query runtime
          - 'execution_id'  : str, Athena query ID for debugging

    Raises:
        RuntimeError: If the query fails - includes Athena's error message.
        TimeoutError: If the query doesn't complete within timeout_s.
    """
    start_response = athena.start_query_execution(
        QueryString=query,
        WorkGroup=workgroup,
        ResultConfiguration={"OutputLocation": output_location},
    )
    execution_id: str = start_response["QueryExecutionId"]

    # Poll until done
    elapsed: float = 0.0
    while elapsed < timeout_s:
        status_response = athena.
↪get_query_execution(QueryExecutionId=execution_id)
        state = status_response["QueryExecution"]["Status"]["State"]

        if state == "SUCCEEDED":
            break
        if state in ("FAILED", "CANCELLED"):
            reason = status_response["QueryExecution"]["Status"].get(
                "StateChangeReason", "no reason given"
            )
            raise RuntimeError(
                f"Athena query {state} (id={execution_id}):\n  {reason}"
            )
        time.sleep(poll_interval_s)
        elapsed += poll_interval_s
    else:
        raise TimeoutError(
            f"Athena query did not finish within {timeout_s}s␣
↪(id={execution_id})"
        )

    # Extract statistics
    stats = status_response["QueryExecution"]["Statistics"]
    scanned_bytes: int = stats.get("DataScannedInBytes", 0)
    runtime_ms: int = stats.get("TotalExecutionTimeInMillis", 0)

    # Fetch results (for small result sets - paginate for large)
```

```
    results = athena.get_query_results(QueryExecutionId=execution_id)
    rows: list[list[str]] = [
        [col.get("VarCharValue", "") for col in row["Data"]]
        for row in results["ResultSet"]["Rows"]
    ]

    return {
        "rows": rows,
        "scanned_bytes": scanned_bytes,
        "runtime_ms": runtime_ms,
        "execution_id": execution_id,
    }


#   Step 3: Query 1 - PA baseline row count (validates partition pruning)
print("\n" + "=" * 70)
print("Query 1: Row count for PA + upgrade=0 (baseline)")
print("=" * 70)

query_1 = f"""
SELECT COUNT(*) AS row_count
FROM "{DB_NAME}".{TS_TABLE}
WHERE state = 'PA' AND upgrade = 0
"""

result_1 = run_athena_query(query_1)
row_count_str = result_1["rows"][1][0]  # rows[0] is headers, rows[1] is data
row_count = int(row_count_str)
scanned_mb = result_1["scanned_bytes"] / 1e6

print(f"  Row count        : {row_count:,d}")
print(f"  Expected         : 807,742,080")
print(f"  Match            : {' ' if row_count == 807_742_080 else '␣
 ↪MISMATCH'}")
print(f"  Data scanned     : {scanned_mb:.2f} MB  (expected ~0 MB for partition␣
 ↪pruning)")
print(f"  Runtime          : {result_1['runtime_ms'] / 1000:.2f} s")

if scanned_mb > 10:
    print(f"    WARNING: Data scanned is higher than expected.")
    print(f"    Partition pruning may not be working - investigate before␣
 ↪scaling.")


#   Step 4: Query 2 - Unique buildings in Allegheny County
# NOTE: The county column name is likely 'in.county' (GISJOIN format like␣
 ↪G4200030)
```

```
# but may vary. Adjust if the schema printout in Step 1 shows a different name.
print("\n" + "=" * 70)
print("Query 2: Unique buildings in Allegheny County, PA + upgrade=0")
print("=" * 70)

# Metadata table has one row per bldg_id with all characteristics.
# Timeseries table is partitioned by (upgrade, state) only - no county column.
# Reference: 2,434 buildings in Allegheny County, PA, baseline (upgrade=0).
METADATA_TABLE: str = "resstock_amy2018_release_1_1_metadata"

query_2 = f"""
SELECT COUNT(DISTINCT bldg_id) AS n_buildings
FROM "{DB_NAME}".{METADATA_TABLE}
WHERE "in.state" = 'PA'
  AND "in.county" = 'G4200030'
"""

try:
    result_2 = run_athena_query(query_2)
    n_buildings = int(result_2["rows"][1][0])
    scanned_mb_2 = result_2["scanned_bytes"] / 1e6

    print(f"  Unique buildings : {n_buildings:,d}")
    print(f"  Expected         : 2,434")
    print(f"  Match            : {' ' if n_buildings == 2_434 else ' ␣
 ↪MISMATCH'}")
    print(f"  Data scanned     : {scanned_mb_2:.2f} MB")
    print(f"  Runtime          : {result_2['runtime_ms'] / 1000:.2f} s")
except RuntimeError as e:
    # Most likely cause: the county column isn't named 'in.county'.
    print(f"    Query failed: {e}")
    print(f"\n  Check the schema printout above for the actual county column␣
 ↪name.")
    print(f"  Candidates often include: 'in.county', 'county', 'in.
 ↪county_name'")

print("\n  Verification complete")
```

```
 Timeseries table schema (119 columns)
 timestamp                                                    timestamp
 out.electricity.ceiling_fan.energy_consumption               double
 out.electricity.ceiling_fan.energy_consumption_intensity     double
 out.electricity.clothes_dryer.energy_consumption             double
 out.electricity.clothes_dryer.energy_consumption_intensity   double
 out.electricity.clothes_washer.energy_consumption            double
 out.electricity.clothes_washer.energy_consumption_intensity  double
 out.electricity.cooling_fans_pumps.energy_consumption        double
```

```
out.electricity.cooling_fans_pumps.energy_consumption_intensity double
out.electricity.cooling.energy_consumption                      double
out.electricity.cooling.energy_consumption_intensity            double
out.electricity.dishwasher.energy_consumption                   double
out.electricity.dishwasher.energy_consumption_intensity         double
out.electricity.freezer.energy_consumption                      double
out.electricity.freezer.energy_consumption_intensity            double
out.electricity.heating_fans_pumps.energy_consumption           double
out.electricity.heating_fans_pumps.energy_consumption_intensity double
out.electricity.heating_hp_bkup.energy_consumption              double
out.electricity.heating_hp_bkup.energy_consumption_intensity    double
out.electricity.heating.energy_consumption                      double
out.electricity.heating.energy_consumption_intensity            double
out.electricity.hot_tub_heater.energy_consumption               double
out.electricity.hot_tub_heater.energy_consumption_intensity     double
out.electricity.hot_tub_pump.energy_consumption                 double
out.electricity.hot_tub_pump.energy_consumption_intensity       double
out.electricity.hot_water.energy_consumption                    double
out.electricity.hot_water.energy_consumption_intensity          double
out.electricity.lighting_exterior.energy_consumption            double
out.electricity.lighting_exterior.energy_consumption_intensity double
out.electricity.lighting_garage.energy_consumption              double
out.electricity.lighting_garage.energy_consumption_intensity    double
out.electricity.lighting_interior.energy_consumption            double
out.electricity.lighting_interior.energy_consumption_intensity double
out.electricity.mech_vent.energy_consumption                    double
out.electricity.mech_vent.energy_consumption_intensity          double
out.electricity.plug_loads.energy_consumption                   double
out.electricity.plug_loads.energy_consumption_intensity         double
out.electricity.pool_heater.energy_consumption                  double
out.electricity.pool_heater.energy_consumption_intensity        double
out.electricity.pool_pump.energy_consumption                    double
out.electricity.pool_pump.energy_consumption_intensity          double
out.electricity.pv.energy_consumption                           double
out.electricity.pv.energy_consumption_intensity                 double
out.electricity.range_oven.energy_consumption                   double
out.electricity.range_oven.energy_consumption_intensity         double
out.electricity.refrigerator.energy_consumption                 double
out.electricity.refrigerator.energy_consumption_intensity       double
out.electricity.well_pump.energy_consumption                    double
out.electricity.well_pump.energy_consumption_intensity          double
out.electricity.net.energy_consumption                          double
out.electricity.net.energy_consumption_intensity                double
out.electricity.total.energy_consumption                        double
out.electricity.total.energy_consumption_intensity              double
bldg_id                                                         bigint

(Full column list available in `columns` variable - 119 total)
```

```
======================================================================
Query 1: Row count for PA + upgrade=0 (baseline)
======================================================================
  Row count        : 807,742,080
  Expected         : 807,742,080
  Match            : 
  Data scanned     : 0.00 MB  (expected ~0 MB for partition pruning)
  Runtime          : 5.68 s

======================================================================
Query 2: Unique buildings in Allegheny County, PA + upgrade=0
======================================================================
  Unique buildings : 2,434
  Expected         : 2,434
  Match            : 
  Data scanned     : 41.15 MB
  Runtime          : 1.98 s

 Verification complete
```

[20]:
```python
# P2 Validation Summary (compact)
print(f"Query 1 row_count   : {row_count:,d}  expected 807,742,080  {' ' if␣
 ↪row_count == 807_742_080 else ' '}")
print(f"Query 1 scanned MB  : {scanned_mb:.2f}")
print(f"Query 2 n_buildings : {n_buildings:,d}  expected 2,434  {' ' if␣
 ↪n_buildings == 2_434 else ' '}")
print(f"\n Pre-flight P2 validated" if (row_count == 807_742_080 and␣
 ↪n_buildings == 2_434) else "\n P2 FAILED")
```

```
Query 1 row_count   : 807,742,080  expected 807,742,080
Query 1 scanned MB  : 0.00
Query 2 n_buildings : 2,434  expected 2,434

 Pre-flight P2 validated
```

[21]:
```python
# ============================================================================
# Step 2 Finalization: Column Name Constants
# ============================================================================
# Purpose: Freeze the column names discovered in the schema printout as named
# constants. Downstream steps import these, so if ResStock ever renames a
# column the fix is localized to this cell.
#
# WHY we split metadata-only vs timeseries columns:
#   The timeseries table has 119 columns but NO geography (no county, no state
#   as a data column - state is a partition key). County, state, census region,
#   sampling weight, and applicability all live in the metadata table.
#   Downstream queries need to know which table to hit for which column.
```

```python
# ----- Timeseries table columns (resstock_amy2018_release_1_1_by_state) -----
BLDG_ID_COL: str = "bldg_id"                                    # bigint
TIMESTAMP_COL: str = "timestamp"                                # timestamp type
ELEC_TOTAL_COL: str = "out.electricity.total.energy_consumption"  # double, kWh/
 ↪h

# Partition keys (also available as WHERE-clause filters, not SELECT columns)
UPGRADE_PARTITION_KEY: str = "upgrade"                          # int, 0 = baseline
STATE_PARTITION_KEY: str = "state"                              # string, 2-char

# ----- Metadata table columns (resstock_amy2018_release_1_1_metadata) -----
METADATA_TABLE: str = "resstock_amy2018_release_1_1_metadata"
COUNTY_COL: str = "in.county"                                   # GISJOIN format
STATE_COL: str = "in.state"                                     # 2-char state code
WEIGHT_COL: str = "weight"                                      # ~240 per bldg

# ----- Reference values for Allegheny County validation -----
SAMPLING_WEIGHT: float = 240.0                                  # simulated → real␣
 ↪homes
TEST_FIPS: str = "42003"
TEST_GISJOIN: str = "G4200030"

# Sanity check: confirm the named columns actually exist in the schema printout
_ts_col_names = {c["Name"] for c in columns}
_required_ts_cols = {BLDG_ID_COL, TIMESTAMP_COL, ELEC_TOTAL_COL}
_missing = _required_ts_cols - _ts_col_names
if _missing:
    raise KeyError(
        f"Expected timeseries columns not found in schema: {_missing}\n"
        f"Available columns include: {sorted(list(_ts_col_names))[:10]}..."
    )
print(f"  Step 2 finalization: {len(_required_ts_cols)} TS columns confirmed")
print(f"   BLDG_ID_COL    : {BLDG_ID_COL}")
print(f"   TIMESTAMP_COL  : {TIMESTAMP_COL}")
print(f"   ELEC_TOTAL_COL : {ELEC_TOTAL_COL}")
print(f"   METADATA_TABLE : {METADATA_TABLE}")
print(f"   TEST_FIPS      : {TEST_FIPS}")
print(f"   TEST_GISJOIN   : {TEST_GISJOIN}")
```

```
 Step 2 finalization: 3 TS columns confirmed
  BLDG_ID_COL    : bldg_id
  TIMESTAMP_COL  : timestamp
  ELEC_TOTAL_COL : out.electricity.total.energy_consumption
  METADATA_TABLE : resstock_amy2018_release_1_1_metadata
  TEST_FIPS      : 42003
  TEST_GISJOIN   : G4200030
```

[ ]:

Step 3: County Geography Mapping — FIPS / GISJOIN to Shapefile

ResStock EUSS uses either FIPS codes or GISJOIN identifiers for county-level geography. We need to confirm which identifier is present and map it to standard Census TIGER/Line shapefiles for choropleth visualization.

GISJOIN format: `G` + state FIPS (2 digits, zero-padded) + `0` + county FIPS (3 digits, zero-padded). Example: Allegheny County PA = `G4200030`.

Standard FIPS format: 5-digit string. Example: Allegheny County PA = `42003`.

Goal of this step: Build a lookup table `county_geo_df` that maps ResStock's county identifier → FIPS → county name → state, so all downstream results can be joined to shapefiles.

Test case: Confirm that Allegheny County, PA appears with FIPS `42003`.

[22]:
```
# CONTEXT: ResStock EUSS 2022.1.1 uses county-level geography identifiers (FIPS␣
 ↪or
# GISJOIN) to tag each building. We need to map these to standard Census TIGER/
 ↪Line
# county shapefiles for visualization. Python 3.11, geopandas already imported.
#
# TASK: Build a county FIPS → name → shapefile geometry lookup for Step 10
# visualization. NOT needed for Steps 4-9 (those work directly from in.county
# GISJOIN values in the TARE DataFrame).
#
# INPUTS:
#   - County shapefile: tl_2025_us_county/tl_2025_us_county.shp
#   - TEST_FIPS from Step 2 constants
# OUTPUTS:
#   - `county_geo_df` (pd.DataFrame): [fips_5digit, county_name, state_fips]
#   - `gdf_counties` (gpd.GeoDataFrame): shapefile joined to county_geo_df

import os
from typing import Final
import geopandas as gpd

# County shapefile path (separate from the state-level SHAPEFILE_PATH)
COUNTY_SHAPEFILE_PATH: str = os.path.join(
    PROJECT_ROOT, "cmu_tare_model", "data", "electricity_ng_price_ratio",
    "tl_2025_us_county", "tl_2025_us_county.shp"
)

# TIGER/Line county shapefiles: GEOID = 5-digit FIPS, NAME = county name,
# STATEFP = 2-digit state FIPS (no state abbreviation in county shapefiles).
TIGER_FIPS_COL: Final[str] = "GEOID"
```

```
TIGER_NAME_COL: Final[str] = "NAME"
TIGER_STATEFP_COL: Final[str] = "STATEFP"

# Load the county shapefile
gdf_counties_raw: gpd.GeoDataFrame = gpd.read_file(COUNTY_SHAPEFILE_PATH)

# Validate expected columns exist
_expected_cols = {TIGER_FIPS_COL, TIGER_NAME_COL, TIGER_STATEFP_COL}
_missing = _expected_cols - set(gdf_counties_raw.columns)
if _missing:
    raise KeyError(
        f"TIGER county shapefile missing expected columns: {_missing}\n"
        f"  Available: {sorted(gdf_counties_raw.columns.tolist())}\n"
        f"  Path: {COUNTY_SHAPEFILE_PATH}"
    )

# Build the plain lookup DataFrame
county_geo_df: pd.DataFrame = gdf_counties_raw[
    [TIGER_FIPS_COL, TIGER_NAME_COL, TIGER_STATEFP_COL]
].rename(
    columns={
        TIGER_FIPS_COL: "fips_5digit",
        TIGER_NAME_COL: "county_name",
        TIGER_STATEFP_COL: "state_fips",
    }
).copy()
county_geo_df["fips_5digit"] = county_geo_df["fips_5digit"].astype(str).str.
 ↪zfill(5)

# GeoDataFrame with renamed columns for downstream merges
gdf_counties: gpd.GeoDataFrame = gdf_counties_raw.rename(
    columns={
        TIGER_FIPS_COL: "fips_5digit",
        TIGER_NAME_COL: "county_name",
        TIGER_STATEFP_COL: "state_fips",
    }
)[["fips_5digit", "county_name", "state_fips", "geometry"]]
gdf_counties["fips_5digit"] = gdf_counties["fips_5digit"].astype(str).str.
 ↪zfill(5)

# Validation: Allegheny County, PA (FIPS 42003) must be present
test_row = county_geo_df[county_geo_df["fips_5digit"] == TEST_FIPS]
if test_row.empty:
    raise ValueError(
        f"Test county FIPS {TEST_FIPS} (Allegheny, PA) not found in shapefile.
 ↪\n"
```

```
        f"  This blocks the Step 7 validation case. Check COUNTY_SHAPEFILE_PATH.
 ↪"
    )
print(f"  county_geo_df: {len(county_geo_df):,d} counties")
print(f"  Test county: {test_row.iloc[0].to_dict()}")
print(f"  gdf_counties CRS: {gdf_counties.crs}")
```

```
  county_geo_df: 3,235 counties
  Test county: {'fips_5digit': '42003', 'county_name': 'Allegheny',
'state_fips': '42'}
  gdf_counties CRS: EPSG:4269
```

[ ]:

Step 4: Extract Adopter Building IDs from TARE Results

The TARE model assigns each building (`bldg_id`) to an adoption tier: - **Tier 1** — Private NPV positive (saves money without incentives) - **Tier 2** — Private NPV positive only with IRA rebates - **Tier 3** — Requires public benefits (social cost of carbon) to be justified - **Tier 4** — No adoption pathway under current conditions

For the **economically-constrained scenario**, adopters = Tier 1 + Tier 2. For the **100% adoption counterfactual**, adopters = all filtered building IDs.

This step extracts both sets of building IDs, organized by county (FIPS), for use in the timeseries queries in Steps 5–6.

Output structure:

```
adopter_ids_by_county = {
    "42003": {                      # FIPS for Allegheny County, PA
        "tier1": [101, 204, ...],
        "tier2": [305, 412, ...],
        "constrained": [101, 204, 305, 412, ...],  # tier1 + tier2
        "all_filtered": [101, 204, ..., 99999],    # 100% adoption
    }
}
```

[23]:

```
# CONTEXT: TARE model results are stored in DATAFRAMES_BY_MP (loaded in Step␣
 ↪0c).
# Each row is a building (bldg_id index) with columns for private NPV, tier␣
 ↪assignment,
# county geography, and applicability. We need to extract adopter building IDs␣
 ↪grouped
# by county for both the constrained and 100% adoption scenarios.
#
# TASK:
#   1. For the selected MP (selected_mps[0]), access␣
 ↪DATAFRAMES_BY_MP[mp]['fixed_base'].
```

```
#   2. Identify the tier assignment column(s). Print available columns if␣
 ↪uncertain.
#   3. Extract two sets of bldg_ids per county:
#      (a) `constrained`: Tier 1 + Tier 2 adopters
#      (b) `all_filtered`: all buildings that passed the occupancy/SF/
 ↪applicable filters
#   4. Build `adopter_ids_by_county` dict keyed by FIPS (from county_geo_df␣
 ↪mapping).
#   5. Print a summary: total adopters vs total buildings for the test county␣
 ↪(FIPS 42003).
#
# INPUTS:
#   - `DATAFRAMES_BY_MP` (Dict[int, Dict]): TARE results loaded in Step 0c
#   - `selected_mps` (List[int]): MP numbers selected in Step 0b
#   - county FIPS derived directly from `in.county` via GISJOIN conversion
# OUTPUTS:
#   - `adopter_ids_by_county` (Dict[str, Dict[str, List[int]]]):
#     keyed by FIPS → {"tier1", "tier2", "constrained", "all_filtered"}
#   - `primary_mp` (int): the single selected MP (selected_mps[0])
# CONSTRAINTS: Type hints on all helper functions. Google/NumPy docstrings.
#              Fail fast if tier column names are not found - print available␣
 ↪columns.

from cmu_tare_model.utils.column_names import create_adoption_col

#   Configuration
primary_mp: int = selected_mps[0]
DISCOUNT_RATE_KEY: str = "fixed_base"
RCM_MODEL_KEY: str = "inmap"

# Tier value strings (must match determine_adoption_potential_sensitivity.py)
TIER_1_VALUE: str = "Tier 1: Feasible"
TIER_2_VALUE: str = "Tier 2: Feasible vs. Alternative"


def gisjoin_to_fips(gisjoin: str) -> str:
    """Convert a GISJOIN county identifier to a 5-digit FIPS code.

    GISJOIN format: G + 2-digit state FIPS + 0 + 3-digit county FIPS.
    Example: 'G4200030' → '42003'.

    Args:
        gisjoin: GISJOIN string from the EUSS ``in.county`` column.

    Returns:
        5-digit county FIPS code as a string.
```

```
    Raises:
        ValueError: If *gisjoin* is shorter than 7 characters.
    """
    if len(gisjoin) < 7:
        raise ValueError(
            f"GISJOIN string too short ({len(gisjoin)} chars): '{gisjoin}'. "
            f"Expected format 'G##0###' (7 chars)."
        )
    return gisjoin[1:3] + gisjoin[4:7]


def find_adoption_column(df: pd.DataFrame, mp: int, cost_scenario: str) -> str:
    """Locate the adoption-tier column in a TARE output DataFrame.

    Builds the expected column name using ``create_adoption_col`` for the
    IRA-reference scenario with central SCC, InMAP, ACS, and the
    ``fixed_base`` discount rate.  Falls back to a fuzzy search if the
    exact column is absent.

    Args:
        df: TARE output DataFrame (one row per building).
        mp: Measure-package number (e.g. 3 or 4).
        cost_scenario: REMDB cost scenario key (e.g. ``'v4MID'``).

    Returns:
        The matched column name string.

    Raises:
        KeyError: If no matching adoption column is found.  The error
            message includes *all* column names that contain 'adoption'
            so the caller can diagnose the mismatch.
    """
    expected = create_adoption_col(
        scenario_prefix=f"iraRef_mp{mp}_",
        category="heating",
        column_type="adoption",
        cost_scenario=cost_scenario,
        method_suffix=f"_{DISCOUNT_RATE_KEY}",
        scc_assumption="central",
        rcm_model=RCM_MODEL_KEY,
        cr_function="acs",
    )
    if expected in df.columns:
        return expected

    # Fuzzy fallback: find any column containing 'adoption'
    adoption_candidates = [c for c in df.columns if "adoption" in c.lower()]
```

```
    if adoption_candidates:
        raise KeyError(
            f"Expected adoption column '{expected}' not found.\n"
            f"  Candidates containing 'adoption' ({len(adoption_candidates)}):
 ↪\n"
            + "\n".join(f"    • {c}" for c in adoption_candidates)
        )
    raise KeyError(
        f"Expected adoption column '{expected}' not found, "
        f"and no columns containing 'adoption' exist.\n"
        f"  Available columns ({len(df.columns)}):\n"
        + "\n".join(f"    • {c}" for c in sorted(df.columns))
    )


def extract_adopter_ids(
    df_tare: pd.DataFrame,
    adoption_col: str,
) -> dict[str, dict[str, list[int]]]:
    """Build per-county adopter ID dictionary from a TARE output DataFrame.

    For each county (identified via the ``county`` GISJOIN column or
    ``county_fips`` integer column), extracts building IDs for Tier 1,
    Tier 2, constrained (T1 + T2), and all filtered buildings.

    Args:
        df_tare: TARE output DataFrame with ``bldg_id`` as index,
            containing ``county`` (GISJOIN) or ``county_fips`` (int),
            and the specified *adoption_col*.
        adoption_col: Name of the adoption-tier column.

    Returns:
        Nested dict keyed by 5-digit FIPS string → sub-dict with keys
        ``'tier1'``, ``'tier2'``, ``'constrained'``, ``'all_filtered'``,
        each mapping to a list of ``bldg_id`` integers.

    Raises:
        KeyError: If county column or *adoption_col* is missing from *df_tare*.
    """
    # Detect which county column is available
    if "county" in df_tare.columns:
        county_col_name = "county"
    elif "in.county" in df_tare.columns:
        county_col_name = "in.county"
    else:
        raise KeyError(
            f"Neither 'county' nor 'in.county' found in TARE DataFrame.\n"
```

```
            f"  Available columns containing 'county': "
            f"{[c for c in df_tare.columns if 'county' in c.lower()]}"
        )
    required_cols = {county_col_name, adoption_col}
    missing = required_cols - set(df_tare.columns)
    if missing:
        raise KeyError(
            f"Missing column(s) {missing} in TARE DataFrame.\n"
            f"  Available columns:\n"
            + "\n".join(f"    • {c}" for c in sorted(df_tare.columns))
        )

    # Derive FIPS from GISJOIN
    df_work = df_tare[[county_col_name, adoption_col]].copy()
    df_work["county_fips"] = df_work[county_col_name].apply(gisjoin_to_fips)

    result: dict[str, dict[str, list[int]]] = {}

    for fips, grp in df_work.groupby("county_fips"):
        bldg_ids = grp.index.tolist()
        tier_vals = grp[adoption_col]

        tier1_ids = grp.index[tier_vals == TIER_1_VALUE].tolist()
        tier2_ids = grp.index[tier_vals == TIER_2_VALUE].tolist()

        result[str(fips)] = {
            "tier1": tier1_ids,
            "tier2": tier2_ids,
            "constrained": tier1_ids + tier2_ids,
            "all_filtered": bldg_ids,
        }

    return result


#   1. Access the TARE DataFrame
print(f"Primary MP: {primary_mp}")
print(f"Discount rate key: {DISCOUNT_RATE_KEY}")
print(f"RCM model key: {RCM_MODEL_KEY}")

df_tare_nested = DATAFRAMES_BY_MP[primary_mp][DISCOUNT_RATE_KEY]

# Handle nesting: DATAFRAMES_BY_MP[mp][dr_key] may be {rcm_model: DataFrame}
if isinstance(df_tare_nested, dict):
    if RCM_MODEL_KEY not in df_tare_nested:
        raise KeyError(
            f"RCM model '{RCM_MODEL_KEY}' not found. "
```

```
            f"Available: {list(df_tare_nested.keys())}"
        )
    df_tare: pd.DataFrame = df_tare_nested[RCM_MODEL_KEY]
    print(f"  Accessed␣
 ↪DATAFRAMES_BY_MP[{primary_mp}]['{DISCOUNT_RATE_KEY}']['{RCM_MODEL_KEY}']")
else:
    df_tare = df_tare_nested
    print(f"  Accessed DATAFRAMES_BY_MP[{primary_mp}]['{DISCOUNT_RATE_KEY}']␣
 ↪(direct DataFrame)")

print(f"  Shape: {df_tare.shape}")
print(f"  Index name: {df_tare.index.name}")

#   2. Find the adoption column
# Try each cost scenario in priority order until one matches.
adoption_col: str | None = None
for cs in REMDB_COST_SCENARIO_KEYS:
    try:
        adoption_col = find_adoption_column(df_tare, primary_mp, cs)
        print(f"\n  Adoption column found (cost_scenario='{cs}'):\n ␣
 ↪{adoption_col}")
        break
    except KeyError:
        continue

if adoption_col is None:
    # Final attempt - raise with full diagnostics from the first cost scenario
    adoption_col = find_adoption_column(df_tare, primary_mp,␣
 ↪REMDB_COST_SCENARIO_KEYS[0])

# Print tier distribution
tier_counts = df_tare[adoption_col].value_counts()
print(f"\n  Tier distribution (MP{primary_mp})  ")
for tier_val, count in tier_counts.items():
    print(f"  {tier_val:<45s}  {count:>7,d}")
print(f"  {'TOTAL':<45s}  {tier_counts.sum():>7,d}")

#   3. Extract adopter IDs by county
adopter_ids_by_county: dict[str, dict[str, list[int]]] = extract_adopter_ids(
    df_tare, adoption_col
)

n_counties = len(adopter_ids_by_county)
total_constrained = sum(len(v["constrained"]) for v in adopter_ids_by_county.
 ↪values())
total_all = sum(len(v["all_filtered"]) for v in adopter_ids_by_county.values())
print(f"\n  Adopter summary  ")
```

```
print(f"  Counties           : {n_counties:,d}")
print(f"  Constrained (T1+T2): {total_constrained:,d}")
print(f"  All filtered       : {total_all:,d}")

#   4. Test case: Allegheny County, PA (FIPS 42003)
TEST_FIPS = "42003"
if TEST_FIPS in adopter_ids_by_county:
    ac = adopter_ids_by_county[TEST_FIPS]
    print(f"\n  Allegheny County (FIPS {TEST_FIPS})  ")
    print(f"  Tier 1       : {len(ac['tier1']):,d}")
    print(f"  Tier 2       : {len(ac['tier2']):,d}")
    print(f"  Constrained  : {len(ac['constrained']):,d}")
    print(f"  All filtered : {len(ac['all_filtered']):,d}")
else:
    print(f"\n  FIPS {TEST_FIPS} (Allegheny County, PA) not found in results.")
    sample_fips = list(adopter_ids_by_county.keys())[:10]
    print(f"  Available FIPS (first 10): {sample_fips}")

print("\n  Step 4 COMPLETE")
```

```
Primary MP: 3
Discount rate key: fixed_base
RCM model key: inmap
  Accessed DATAFRAMES_BY_MP[3]['fixed_base']['inmap']
  Shape: (331531, 271)
  Index name: bldg_id

  Adoption column found (cost_scenario='v3'):
  iraRef_mp3_heating_adoption_central_inmap_acs_v3_fixed_base

  Tier distribution (MP3)
  Tier 4: Averse                                 151,340
  Tier 3: Subsidy-Dependent Feasibility          106,932
  Tier 1: Feasible                                16,650
  Tier 2: Feasible vs. Alternative                16,453
  N/A: Invalid Baseline Fuel/Tech                    183
  TOTAL                                          291,558

  Adopter summary
  Counties           : 3,098
  Constrained (T1+T2): 33,103
  All filtered       : 331,531

  Allegheny County (FIPS 42003)
  Tier 1       : 64
  Tier 2       : 29
  Constrained  : 93
  All filtered : 1,610
```

```
Step 4 COMPLETE
```

[ ]:

Step 5: Test Query — Baseline Timeseries for Allegheny County (FIPS 42003)

Before building the full pipeline, validate the query pattern on a single county. Allegheny County, PA (FIPS 42003) is the primary case study.

What this step produces: - `df_ts_baseline_allegheny`: shape (8760 × N_buildings) or pre-aggregated (8760,) containing hourly baseline (MP0) electricity consumption for all buildings in county

Design decision to resolve here: - Query strategy A: Pull all individual building timeseries → aggregate locally in pandas (higher Athena cost, more flexible locally) - Query strategy B: Push aggregation into Athena SQL → pull only the hourly sum (lower cost, less flexible for adopter/non-adopter split)

Recommendation: Start with Strategy A on Allegheny County only to validate the schema, then decide whether to push aggregation to Athena for the national loop.

[24]:
```
# CONTEXT: Testing BuildStockQuery against the OEDI EUSS 2022.1.1 timeseries␣
 ↪Athena
# table for a single county (Allegheny County, PA, FIPS 42003) before scaling.
# We want hourly baseline (MP0) electricity consumption for all buildings in␣
 ↪this county.
#
# TASK:
#   1. Filter `all_filtered` building IDs for FIPS 42003 from␣
 ↪`adopter_ids_by_county`.
#   2. Query the EUSS baseline timeseries table via BuildStockQuery for those␣
 ↪bldg_ids.
#      Use `BLDG_ID_COL`, `TIMESTAMP_COL`, `ELEC_TOTAL_COL` confirmed in Step 2.
#   3. Return a DataFrame `df_ts_baseline_allegheny` with columns:
#      [bldg_id, hour (1-8760), baseline_kwh].
#   4. Print: shape, memory usage, min/max hour, min/max kWh. Sample 5 rows.
#   5. Time the query and print elapsed seconds - we'll use this to estimate␣
 ↪national cost.
#
# INPUTS:
#   - `rsq` (ResStockQuery): initialized BuildStockQuery object
#   - `adopter_ids_by_county` (dict): from Step 4
#   - `BLDG_ID_COL`, `TIMESTAMP_COL`, `ELEC_TOTAL_COL` (str): from Step 2
# OUTPUTS:
#   - `df_ts_baseline_allegheny` (pd.DataFrame): [bldg_id, hour, baseline_kwh]
#   - `query_time_s` (float): elapsed query time in seconds
```

```
# CONSTRAINTS: Type hints. Wrap query in try/except - print Athena error␣
 ↪clearly.
#              Do not pull more columns than needed (minimize scan cost).

import time
from typing import cast

#  1. BSQ initialization - skip if incompatible with EUSS schema
# BSQ expects 'building_id' but EUSS 2022.1.1 uses 'bldg_id'. Rather than
# patching BSQ internals, we use the raw SQL fallback path via␣
 ↪run_athena_query().
# This was anticipated in the plan as the "hyphenated db-name / schema" risk.
bsq = None  # placeholder - not usable for this dataset
print("  BSQ skipped (EUSS uses 'bldg_id', BSQ expects 'building_id')")
print("  Using run_athena_query() fallback for all timeseries queries.")

#  2. Pull the bldg_id list for Allegheny County from the TARE output
allegheny_bldg_ids: list[int] = adopter_ids_by_county[TEST_FIPS]["all_filtered"]
if not allegheny_bldg_ids:
    raise ValueError(
        f"No filtered bldg_ids for FIPS {TEST_FIPS} in adopter_ids_by_county. "
        f"Re-check Step 4 outputs."
    )
print(f"\n  Allegheny bldg_ids (all_filtered): {len(allegheny_bldg_ids):,d}")

#  3. Query baseline timeseries via raw SQL
_id_list_sql = ", ".join(str(i) for i in allegheny_bldg_ids)
_sql = f"""
SELECT bldg_id, timestamp, "{ELEC_TOTAL_COL}" AS kwh
FROM "{DB_NAME}".{TS_TABLE}
WHERE state = 'PA'
  AND upgrade = 0
  AND bldg_id IN ({_id_list_sql})
"""
print(f"\n Querying baseline timeseries (upgrade=0, {len(allegheny_bldg_ids)}␣
 ↪buildings)...")
t_start = time.perf_counter()

# Start query and wait for completion
start_response = athena.start_query_execution(
    QueryString=_sql,
    WorkGroup=WORKGROUP_NAME,
    ResultConfiguration={"OutputLocation": ATHENA_RESULTS_S3},
)
execution_id: str = start_response["QueryExecutionId"]

# Poll until done
```

```
elapsed: float = 0.0
timeout_s: float = 600.0
while elapsed < timeout_s:
    status_response = athena.get_query_execution(QueryExecutionId=execution_id)
    state = status_response["QueryExecution"]["Status"]["State"]
    if state == "SUCCEEDED":
        break
    if state in ("FAILED", "CANCELLED"):
        reason = status_response["QueryExecution"]["Status"].get(
            "StateChangeReason", "no reason given"
        )
        raise RuntimeError(f"Athena query {state} (id={execution_id}):\n ␣
 ↪{reason}")
    time.sleep(2.0)
    elapsed += 2.0
else:
    raise TimeoutError(f"Athena query did not finish within {timeout_s}s")

stats = status_response["QueryExecution"]["Statistics"]
scanned_gb = stats.get("DataScannedInBytes", 0) / 1e9
print(f"  Query completed in {stats.get('TotalExecutionTimeInMillis', 0)/1000:.
 ↪1f}s, scanned {scanned_gb:.2f} GB")

# Read results from S3 as CSV (much faster than paginating get_query_results␣
 ↪for 14M rows)
output_location =␣
 ↪status_response["QueryExecution"]["ResultConfiguration"]["OutputLocation"]
# Parse S3 URI: s3://bucket/key
_s3_parts = output_location.replace("s3://", "").split("/", 1)
_result_bucket = _s3_parts[0]
_result_key = _s3_parts[1]

import io
_obj = s3.get_object(Bucket=_result_bucket, Key=_result_key)
df_ts_baseline_allegheny: pd.DataFrame = pd.read_csv(
    io.BytesIO(_obj["Body"].read()),
    dtype={"bldg_id": "int64", "kwh": "float64"},
    parse_dates=["timestamp"],
)
df_ts_baseline_allegheny = df_ts_baseline_allegheny.rename(columns={"kwh":␣
 ↪"baseline_kwh"})
query_time_s: float = time.perf_counter() - t_start
print(f"  Query + download returned in {query_time_s:.1f} s")

#  4. Aggregate to hourly resolution (data is 15-min intervals, 35,040/bldg)
# EUSS timeseries stores 15-min data (kWh per 15 min). Sum four intervals →␣
 ↪hourly kWh.
```

```
df_ts_baseline_allegheny["hour_ts"] = df_ts_baseline_allegheny["timestamp"].dt.
  ↪floor("h")
df_ts_baseline_allegheny = (
    df_ts_baseline_allegheny.groupby([BLDG_ID_COL, "hour_ts"], as_index=False)
    .agg(baseline_kwh=("baseline_kwh", "sum"))
    .rename(columns={"hour_ts": TIMESTAMP_COL})
)

# Trim to AMY2018 year only (8,760 hours). The 15-min data may include a partial
# hour at 2019-01-01 00:00 from the last four intervals of Dec 31.
df_ts_baseline_allegheny = df_ts_baseline_allegheny[
    df_ts_baseline_allegheny[TIMESTAMP_COL].dt.year == 2018
].reset_index(drop=True)

# Add hour index (1..8760)
df_ts_baseline_allegheny = df_ts_baseline_allegheny.sort_values(
    [BLDG_ID_COL, TIMESTAMP_COL]
).reset_index(drop=True)
df_ts_baseline_allegheny["hour"] = (
    df_ts_baseline_allegheny.groupby(BLDG_ID_COL).cumcount() + 1
)

#  5. Summary + sanity checks
n_bldgs: int = df_ts_baseline_allegheny[BLDG_ID_COL].nunique()
n_hours_per_bldg = df_ts_baseline_allegheny.groupby(BLDG_ID_COL).size()
print(f"\n  df_ts_baseline_allegheny summary  ")
print(f"  Rows             : {len(df_ts_baseline_allegheny):,d}")
print(f"  Buildings        : {n_bldgs:,d}")
print(f"  Hours/bldg min   : {n_hours_per_bldg.min():,d}")
print(f"  Hours/bldg max   : {n_hours_per_bldg.max():,d}")
print(f"  Memory (MB)      : {df_ts_baseline_allegheny.memory_usage(deep=True).
  ↪sum() / 1e6:.1f}")
print(f"  kWh range        : {df_ts_baseline_allegheny['baseline_kwh'].min():.
  ↪3f}"
      f" to {df_ts_baseline_allegheny['baseline_kwh'].max():.3f}")
print(f"  Query time (s)   : {query_time_s:.2f}")
display(df_ts_baseline_allegheny.head())
```

```
 BSQ skipped (EUSS uses 'bldg_id', BSQ expects 'building_id')
 Using run_athena_query() fallback for all timeseries queries.

 Allegheny bldg_ids (all_filtered): 1,610

Querying baseline timeseries (upgrade=0, 1610 buildings)…
 Query completed in 62.3s, scanned 0.54 GB
 Query + download returned in 543.5 s
```

```
df_ts_baseline_allegheny summary
Rows            : 14,103,600
Buildings       : 1,610
Hours/bldg min  : 8,760
Hours/bldg max  : 8,760
Memory (MB)     : 451.3
kWh range       : 0.111 to 41.683
Query time (s)  : 543.49
```

	bldg_id	timestamp	baseline_kwh	hour
0	491	2018-01-01 00:00:00	0.330	1
1	491	2018-01-01 01:00:00	0.479	2
2	491	2018-01-01 02:00:00	0.389	3
3	491	2018-01-01 03:00:00	0.380	4
4	491	2018-01-01 04:00:00	0.395	5

[25]:
```python
# Step 5b - Fix 8,761 → 8,760 hours (trim 2019-01-01 00:00 boundary hour)
df_ts_baseline_allegheny = df_ts_baseline_allegheny[
    df_ts_baseline_allegheny[TIMESTAMP_COL].dt.year == 2018
].reset_index(drop=True)

# Recompute hour index
df_ts_baseline_allegheny = df_ts_baseline_allegheny.sort_values(
    [BLDG_ID_COL, TIMESTAMP_COL]
).reset_index(drop=True)
df_ts_baseline_allegheny["hour"] = (
    df_ts_baseline_allegheny.groupby(BLDG_ID_COL).cumcount() + 1
)

n_hours_per_bldg = df_ts_baseline_allegheny.groupby(BLDG_ID_COL).size()
print(f"Hours/bldg after trim: {n_hours_per_bldg.min()} - {n_hours_per_bldg.
  ↪max()}")
print(f"Total rows: {len(df_ts_baseline_allegheny):,d}")
```

```
Hours/bldg after trim: 8760 - 8760
Total rows: 14,103,600
```

[ ]:
```python

```

Step 6: Test Query — Upgrade Timeseries (MP3 or MP4) for Allegheny County

Query the post-retrofit timeseries for the same county and building IDs. The upgrade table contains electricity consumption after heat pump installation.

Note: Only buildings where `applicability == True` have valid upgrade data. The `adopter_ids_by_county["42003"]["all_filtered"]` list already respects this filter.

Key column to confirm: Does the upgrade timeseries table have the same schema as the baseline table, or are column names different? Resolve this here before building the aggregation logic in

Step 7.

[26]:
```
# Step 6 - Upgrade timeseries (MP3) for Allegheny County
# Same approach as Step 5: raw SQL → Athena → S3 CSV download → pd.read_csv

import io

print(f"\n Querying upgrade timeseries (upgrade={primary_mp})...")
t_start = time.perf_counter()

_id_list_sql = ", ".join(str(i) for i in allegheny_bldg_ids)
_sql = f"""
SELECT bldg_id, timestamp, "{ELEC_TOTAL_COL}" AS kwh
FROM "{DB_NAME}".{TS_TABLE}
WHERE state = 'PA'
  AND upgrade = {primary_mp}
  AND bldg_id IN ({_id_list_sql})
"""

# Start query and poll
start_response = athena.start_query_execution(
    QueryString=_sql,
    WorkGroup=WORKGROUP_NAME,
    ResultConfiguration={"OutputLocation": ATHENA_RESULTS_S3},
)
execution_id: str = start_response["QueryExecutionId"]

elapsed: float = 0.0
timeout_s: float = 600.0
while elapsed < timeout_s:
    status_response = athena.get_query_execution(QueryExecutionId=execution_id)
    state = status_response["QueryExecution"]["Status"]["State"]
    if state == "SUCCEEDED":
        break
    if state in ("FAILED", "CANCELLED"):
        reason = status_response["QueryExecution"]["Status"].get(
            "StateChangeReason", "no reason given"
        )
        raise RuntimeError(f"Athena query {state} (id={execution_id}):\n ␣
 ↪{reason}")
    time.sleep(2.0)
    elapsed += 2.0
else:
    raise TimeoutError(f"Athena query did not finish within {timeout_s}s")

stats = status_response["QueryExecution"]["Statistics"]
scanned_gb = stats.get("DataScannedInBytes", 0) / 1e9
```

```
print(f"  Query completed in {stats.get('TotalExecutionTimeInMillis', 0)/1000:.
  ↪1f}s, scanned {scanned_gb:.2f} GB")

# Read results from S3 CSV
output_location =␣
  ↪status_response["QueryExecution"]["ResultConfiguration"]["OutputLocation"]
_s3_parts = output_location.replace("s3://", "").split("/", 1)
_result_bucket = _s3_parts[0]
_result_key = _s3_parts[1]

_obj = s3.get_object(Bucket=_result_bucket, Key=_result_key)
df_ts_upgrade_allegheny: pd.DataFrame = pd.read_csv(
    io.BytesIO(_obj["Body"].read()),
    dtype={"bldg_id": "int64", "kwh": "float64"},
    parse_dates=["timestamp"],
)
df_ts_upgrade_allegheny = df_ts_upgrade_allegheny.rename(columns={"kwh":␣
  ↪"retrofit_kwh"})
upgrade_query_time_s: float = time.perf_counter() - t_start
print(f" Query + download returned in {upgrade_query_time_s:.1f} s")

# Aggregate 15-min → hourly
df_ts_upgrade_allegheny["hour_ts"] = df_ts_upgrade_allegheny["timestamp"].dt.
  ↪floor("h")
df_ts_upgrade_allegheny = (
    df_ts_upgrade_allegheny.groupby([BLDG_ID_COL, "hour_ts"], as_index=False)
    .agg(retrofit_kwh=("retrofit_kwh", "sum"))
    .rename(columns={"hour_ts": TIMESTAMP_COL})
)

# Trim to 2018 only
df_ts_upgrade_allegheny = df_ts_upgrade_allegheny[
    df_ts_upgrade_allegheny[TIMESTAMP_COL].dt.year == 2018
].reset_index(drop=True)

# Add hour index
df_ts_upgrade_allegheny = df_ts_upgrade_allegheny.sort_values(
    [BLDG_ID_COL, TIMESTAMP_COL]
).reset_index(drop=True)
df_ts_upgrade_allegheny["hour"] = (
    df_ts_upgrade_allegheny.groupby(BLDG_ID_COL).cumcount() + 1
)

# Schema parity check
baseline_bldgs = set(df_ts_baseline_allegheny[BLDG_ID_COL].unique())
upgrade_bldgs = set(df_ts_upgrade_allegheny[BLDG_ID_COL].unique())
only_in_baseline = baseline_bldgs - upgrade_bldgs
```

```
only_in_upgrade = upgrade_bldgs - baseline_bldgs

n_hours_up = df_ts_upgrade_allegheny.groupby(BLDG_ID_COL).size()
print(f"\n df_ts_upgrade_allegheny summary")
print(f"   Rows            : {len(df_ts_upgrade_allegheny):,d}")
print(f"   Buildings       : {len(upgrade_bldgs):,d}")
print(f"   Hours/bldg      : {n_hours_up.min()} - {n_hours_up.max()}")
print(f"   Baseline bldgs  : {len(baseline_bldgs):,d}")
print(f"   Only in baseline: {len(only_in_baseline):,d}")
print(f"   Only in upgrade : {len(only_in_upgrade):,d}")
print(f"   kWh range       : {df_ts_upgrade_allegheny['retrofit_kwh'].min():.
 ↪3f}"
      f" to {df_ts_upgrade_allegheny['retrofit_kwh'].max():.3f}")
print(f"   Query time (s)  : {upgrade_query_time_s:.2f}")

if only_in_baseline:
    print(f"\n   Note: {len(only_in_baseline):,d} buildings have no upgrade␣
 ↪data.")
    print(f"   They'll use baseline kWh in Step 7.")
if only_in_upgrade:
    raise ValueError(
        f"{len(only_in_upgrade)} bldg_ids in upgrade but not baseline.␣
 ↪Investigate."
    )

display(df_ts_upgrade_allegheny.head())
```

```
Querying upgrade timeseries (upgrade=3)…
 Query completed in 65.3s, scanned 0.57 GB
Query + download returned in 278.2 s

df_ts_upgrade_allegheny summary
  Rows            : 14,103,600
  Buildings       : 1,610
  Hours/bldg      : 8760 - 8760
  Baseline bldgs  : 1,610
  Only in baseline: 0
  Only in upgrade : 0
  kWh range       : 0.137 to 103.596
  Query time (s)  : 278.15
```

	bldg_id	timestamp	retrofit_kwh	hour
0	491	2018-01-01 00:00:00	3.208	1
1	491	2018-01-01 01:00:00	5.068	2
2	491	2018-01-01 02:00:00	2.320	3
3	491	2018-01-01 03:00:00	1.900	4
4	491	2018-01-01 04:00:00	2.605	5

[27]:
```
# Step 6b - Diagnostic: How many buildings does upgrade=3 have for PA?
_diag_sql = f"""
SELECT COUNT(DISTINCT bldg_id) AS n_bldgs, COUNT(*) AS n_rows
FROM "{DB_NAME}".{TS_TABLE}
WHERE state = 'PA' AND upgrade = {primary_mp}
"""
diag_result = run_athena_query(_diag_sql, timeout_s=120.0)
print("Upgrade=3 PA totals:", diag_result["rows"])

# Also check: what upgrades exist?
_diag_sql2 = f"""
SELECT upgrade, COUNT(DISTINCT bldg_id) AS n_bldgs
FROM "{DB_NAME}".{TS_TABLE}
WHERE state = 'PA'
GROUP BY upgrade
ORDER BY upgrade
"""
diag_result2 = run_athena_query(_diag_sql2, timeout_s=120.0)
print("\nAll upgrades for PA:")
for row in diag_result2["rows"]:
    print(f"  {row}")
```

```
Upgrade=3 PA totals: [['n_bldgs', 'n_rows'], ['23052', '807742080']]

All upgrades for PA:
  ['upgrade', 'n_bldgs']
  ['0', '23052']
  ['1', '23052']
  ['2', '23052']
  ['3', '23052']
  ['4', '23052']
  ['5', '23052']
  ['6', '23052']
  ['7', '23052']
  ['8', '23052']
  ['9', '23052']
  ['10', '23052']
```

[28]:
```
# Step 6c - Debug: check Athena query result size for upgrade=3
# Try with just 5 bldg_ids first to see if we get 35,040 rows each
test_ids = allegheny_bldg_ids[:5]
_test_sql = f"""
SELECT bldg_id, COUNT(*) AS n_rows
FROM "{DB_NAME}".{TS_TABLE}
WHERE state = 'PA'
  AND upgrade = {primary_mp}
  AND bldg_id IN ({", ".join(str(i) for i in test_ids)})
GROUP BY bldg_id
```

```
"""
test_result = run_athena_query(_test_sql, timeout_s=120.0)
print("Rows per bldg_id (upgrade=3, 5 test IDs):")
for row in test_result["rows"]:
    print(f"  {row}")

# Also check: how large was the S3 CSV for the full upgrade query?
print(f"\nLength of allegheny_bldg_ids: {len(allegheny_bldg_ids)}")
print(f"SQL length estimate: {len(', '.join(str(i) for i in␣
 ↪allegheny_bldg_ids))} chars")
```

```
Rows per bldg_id (upgrade=3, 5 test IDs):
  ['bldg_id', 'n_rows']
  ['3205', '35040']
  ['640', '35040']
  ['1081', '35040']
  ['491', '35040']
  ['2377', '35040']

Length of allegheny_bldg_ids: 1610
SQL length estimate: 12539 chars
```

[ ]:

Step 7: Compute Scenario Demand Profile — Allegheny County (Test Case)

Applies the adopter/non-adopter mask to produce a scenario hourly demand profile.

Logic (per building, per hour): - If `bldg_id` is in `adopter_ids` → use `retrofit_kwh` (post-HP electricity) - Otherwise → use `baseline_kwh` (existing equipment electricity)

Two profiles produced: 1. **100% adoption:** all filtered buildings adopt → all use `retrofit_kwh` 2. **Constrained adoption:** Tier 1 + Tier 2 adopt; Tier 3 + Tier 4 use baseline

Peak load change = `max(scenario_profile_kw)` − `max(baseline_profile_kw)`

Units note: BuildStockQuery returns kWh per hour. Since each interval is 1 hour, kWh = kW for that hour. Multiply by sampling weight (~240) to convert from simulated units to real-world MW equivalents.

[29]:
```
from typing import Any

def compute_county_scenario_profile(
    df_baseline: pd.DataFrame,
    df_upgrade: pd.DataFrame,
    adopter_bldg_ids: list[int],
    sampling_weight: float = 240.0,
) -> tuple[pd.DataFrame, dict[str, Any]]:
    """Compute hourly baseline and scenario demand profiles for one county.
```

```
    Args:
        df_baseline: Columns [bldg_id, hour, baseline_kwh]. 8,760 rows per␣
↪building.
        df_upgrade: Columns [bldg_id, hour, retrofit_kwh]. May be a subset of␣
↪baseline.
        adopter_bldg_ids: Buildings that adopt the retrofit.
        sampling_weight: Real homes per simulated building (EUSS 2022.1.1 =␣
↪240).

    Returns:
        (df_profile, peak_dict) where df_profile has [hour, baseline_mw,␣
↪scenario_mw, delta_mw].
    """
    adopter_set: set[int] = set(adopter_bldg_ids)
    all_baseline_bldgs: set[int] = set(df_baseline[BLDG_ID_COL].unique())
    upgrade_bldgs: set[int] = set(df_upgrade[BLDG_ID_COL].unique())

    # Effective adopters = those with upgrade data
    adopters_missing_upgrade = adopter_set - upgrade_bldgs
    if adopters_missing_upgrade:
        print(f"  {len(adopters_missing_upgrade):,d} adopter bldg_ids have no␣
↪upgrade data - using baseline.")
    effective_adopters: set[int] = adopter_set & upgrade_bldgs

    # Left-join baseline ← upgrade
    df_merged: pd.DataFrame = df_baseline.merge(
        df_upgrade[[BLDG_ID_COL, "hour", "retrofit_kwh"]],
        on=[BLDG_ID_COL, "hour"],
        how="left",
    )

    # Vectorized adopter mask
    is_effective_adopter = df_merged[BLDG_ID_COL].isin(effective_adopters)
    retrofit_filled = df_merged["retrofit_kwh"].
↪fillna(df_merged["baseline_kwh"])
    df_merged["scenario_kwh"] = np.where(
        is_effective_adopter, retrofit_filled, df_merged["baseline_kwh"]
    )

    # Aggregate across buildings → hourly county profile (MW)
    kwh_to_mw: float = sampling_weight / 1000.0
    df_profile: pd.DataFrame = (
        df_merged.groupby("hour", as_index=False)
        .agg(
            baseline_kwh=("baseline_kwh", "sum"),
```

```
            scenario_kwh=("scenario_kwh", "sum"),
        )
    )
    df_profile["baseline_mw"] = df_profile["baseline_kwh"] * kwh_to_mw
    df_profile["scenario_mw"] = df_profile["scenario_kwh"] * kwh_to_mw
    df_profile["delta_mw"] = df_profile["scenario_mw"] -␣
 ↪df_profile["baseline_mw"]
    df_profile = df_profile[["hour", "baseline_mw", "scenario_mw", "delta_mw"]]

    if len(df_profile) != 8760:
        raise ValueError(
            f"Expected 8,760 hourly rows, got {len(df_profile):,d}. "
            f"Hour range: {df_profile['hour'].min()}..{df_profile['hour'].
 ↪max()}"
        )

    peak_dict: dict[str, Any] = {
        "peak_hour_baseline": int(df_profile.loc[df_profile["baseline_mw"].
 ↪idxmax(), "hour"]),
        "peak_hour_scenario": int(df_profile.loc[df_profile["scenario_mw"].
 ↪idxmax(), "hour"]),
        "baseline_peak_mw": float(df_profile["baseline_mw"].max()),
        "scenario_peak_mw": float(df_profile["scenario_mw"].max()),
        "delta_mw": float(df_profile["scenario_mw"].max() -␣
 ↪df_profile["baseline_mw"].max()),
        "n_adopters": len(effective_adopters),
        "n_total_buildings": len(all_baseline_bldgs),
    }

    return df_profile, peak_dict


# --- Call for Allegheny County ---
print(" Computing 100% adoption profile...")
df_profile_100pct, peak_100pct = compute_county_scenario_profile(
    df_ts_baseline_allegheny,
    df_ts_upgrade_allegheny,
    adopter_bldg_ids=adopter_ids_by_county[TEST_FIPS]["all_filtered"],
    sampling_weight=SAMPLING_WEIGHT,
)

print(" Computing constrained adoption profile (Tier 1+2)...")
df_profile_constrained, peak_constrained = compute_county_scenario_profile(
    df_ts_baseline_allegheny,
    df_ts_upgrade_allegheny,
    adopter_bldg_ids=adopter_ids_by_county[TEST_FIPS]["constrained"],
    sampling_weight=SAMPLING_WEIGHT,
```

```
)

peak_results_allegheny: dict[str, dict[str, Any]] = {
    "100pct": peak_100pct,
    "constrained": peak_constrained,
}

print(f"\n Allegheny peak results (MP{primary_mp})")
for scenario, p in peak_results_allegheny.items():
    print(f"  [{scenario}]")
    print(f"    adopters      : {p['n_adopters']:,d} / {p['n_total_buildings']:
 ↪,d}")
    print(f"    baseline peak : {p['baseline_peak_mw']:.2f} MW @ hour␣
 ↪{p['peak_hour_baseline']}")
    print(f"    scenario peak : {p['scenario_peak_mw']:.2f} MW @ hour␣
 ↪{p['peak_hour_scenario']}")
    print(f"    delta         : {p['delta_mw']:+.2f} MW")

print(f"\n Validation: len(df_profile_100pct) = {len(df_profile_100pct)}")
print(f" Validation: len(df_profile_constrained) =␣
 ↪{len(df_profile_constrained)}")
assert len(df_profile_100pct) == 8760, "100pct profile not 8760 rows!"
assert len(df_profile_constrained) == 8760, "constrained profile not 8760 rows!"
print(" Step 7 PASSED")
```

```
Computing 100% adoption profile…
Computing constrained adoption profile (Tier 1+2)…

Allegheny peak results (MP3)
  [100pct]
    adopters      : 1,610 / 1,610
    baseline peak : 853.94 MW @ hour 4433
    scenario peak : 6524.90 MW @ hour 152
    delta         : +5670.97 MW
  [constrained]
    adopters      : 93 / 1,610
    baseline peak : 853.94 MW @ hour 4433
    scenario peak : 873.78 MW @ hour 140
    delta         : +19.84 MW

Validation: len(df_profile_100pct) = 8760
Validation: len(df_profile_constrained) = 8760
Step 7 PASSED
```

[ ]:

Step 8: Validate — Compare Aggregated Profile Against EUSS Peak Load Columns

EUSS annual/metadata files include individual household peak load values (kW). These provide a within-dataset sanity check before comparing to external benchmarks.

Validation approach: - Sum the EUSS individual peak load values for Allegheny County buildings → county peak (naive sum) - Compare to the profile-derived peak from Step 7 (baseline, 100% adoption) - These will not be identical (naive sum coincident peak), but should be same order of magnitude

External benchmark (after internal check passes): - Compare PA statewide peak load change to Maxim et al. (2024) 2018 vs. 2050 scenarios - Reference OEDI URL from notebook stub: `https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2Fend-us`

Flag: If profile-derived peak differs from EUSS column sum by more than 20%, investigate before scaling to national loop.

[30]:
```
# CONTEXT: Validating our aggregated timeseries-derived peak load against the␣
 ↪EUSS
# annual metadata file's built-in peak load columns for Allegheny County, PA.
# df_baseline (the annual/metadata CSV) is already loaded from Step 1 of the␣
 ↪notebook.
#
# TASK:
#   1. Identify the peak load column(s) in `df_baseline` (annual CSV).
#      Look for columns containing 'peak' or 'max' in the column name - print␣
 ↪candidates.
#   2. Filter df_baseline to Allegheny County buildings (use county_geo_df␣
 ↪mapping).
#   3. Sum the individual peak load values → `naive_county_peak_kw`.
#      (Naive sum is an upper bound - assumes all buildings peak simultaneously.
 ↪)
#   4. Compare to `peak_results_allegheny["baseline_peak_mw"] * 1000` (convert␣
 ↪to kW).
#   5. Compute ratio: naive_sum / profile_derived_peak. Print result with␣
 ↪interpretation.
#   6. Flag with a warning if ratio is outside [0.8, 5.0] (rough sanity bounds).
#
# INPUTS:
#   - `df_baseline` (pd.DataFrame): EUSS annual metadata CSV, loaded in Step 1
#   - `county_geo_df` (pd.DataFrame): county mapping from Step 3
#   - `peak_results_allegheny` (dict): peak load results from Step 7
# OUTPUTS: Printed validation summary. No new DataFrames required.
# CONSTRAINTS: Type hints on any helper. If peak column name is uncertain,
#              print all column names containing 'peak', 'max', or 'load'.

# --- PLACEHOLDER: Opus/Copilot drafts implementation below ---
```

[ ]:

Step 9: National Loop — County-Level Peak Load Table

Run Step 7 validation on Allegheny County BEFORE running this step.

Scales the Allegheny County pipeline across all counties present in `adopter_ids_by_county`.

Design decisions to resolve before running: 1. **Query batching:** Query all buildings for a county in one Athena call, or batch by state first (reduces number of Athena connections)? 2. **Aggregation location:** Push the hourly sum to Athena SQL (cheaper, faster) or pull building-level data and aggregate in pandas (more flexible for adopter mask)? 3. **Checkpoint saving:** Save intermediate results per state to disk in case the loop fails mid-run (strongly recommended for a national run).

Expected output: `df_peak_results_national` with one row per county: `[fips, county_name, state, n_adopters_constrained, n_all_filtered, baseline_peak_mw, scenario_100pct_peak_mw, scenario_constrained_peak_mw, delta_100pct_mw, delta_constrained_mw, peak_hour_100pct, peak_hour_constrained]`

[31]:
```
# CONTEXT: Scaling the county-level peak load pipeline (Steps 5-7) to all␣
 ↪counties
# in the TARE model results. ResStock EUSS 2022.1.1 via AWS Athena /␣
 ↪BuildStockQuery.
# Python 3.11, conda env cmu-tare-model. Allegheny County (FIPS 42003)␣
 ↪validated in Step 7.
#
# TASK: Implement `run_national_peak_load_loop()` that:
#   1. Iterates over all FIPS keys in `adopter_ids_by_county`.
#   2. For each county: queries baseline + upgrade timeseries (reuse logic from␣
 ↪Steps 5-6),
#      calls `compute_county_scenario_profile()` for both adoption scenarios.
#   3. Stores peak load results in a running list → converts to DataFrame at␣
 ↪end.
#   4. Saves a checkpoint file per state (e.g., `peak_results_PA.csv`) so␣
 ↪progress is
#      not lost if the loop fails.
#   5. Prints progress: "County X of N | FIPS {fips} | {county_name}, {state} |␣
 ↪"
#   6. Returns `df_peak_results_national` (pd.DataFrame) with schema described␣
 ↪above.
#
# INPUTS:
#   - `rsq` (ResStockQuery): initialized BuildStockQuery object
#   - `adopter_ids_by_county` (dict): from Step 4
#   - `county_geo_df` (pd.DataFrame): from Step 3
#   - `primary_mp` (int): selected measure package
```

```
#   - `PROJECT_ROOT` (str): from config, for checkpoint file paths
# OUTPUTS:
#   - `df_peak_results_national` (pd.DataFrame): county-level peak load results
# CONSTRAINTS: Type hints. Google/NumPy docstring. Wrap each county query in try/except
#              - log failures to a `failed_counties` list and continue.
#              Do not run without Step 7 validation passing first.

def run_national_peak_load_loop(
    rsq: object,
    adopter_ids_by_county: "dict[str, dict[str, list[int]]]",
    county_geo_df: "pd.DataFrame",
    primary_mp: int,
    project_root: str,
    sampling_weight: float = 240.0,
) -> "pd.DataFrame":
    """
    # --- PLACEHOLDER: Opus/Copilot completes this function ---
    """
    raise NotImplementedError("Step 9 placeholder - implement with Copilot/Opus")
```

[ ]:

Step 10: Export Results for Paper Figures

Exports the national county-level peak load table for use in: - **Figure XX (paper Section 3.6):** County choropleth of peak load change under 100% adoption and economically-constrained adoption, by technology scenario (MP3/MP4) - **Allegheny County case study panel:** Bar chart comparing baseline vs scenario peak across all technology scenarios

Files exported: - `peak_load_results_MP{mp}_national.csv` — full national county table - `peak_load_results_MP{mp}_allegheny.csv` — Allegheny County only (for case study)

[32]:
```
# CONTEXT: Exporting national county-level peak load results for paper figures.
# Results are in `df_peak_results_national` from Step 9.
#
# TASK:
#   1. Define OUTPUT_DIR using PROJECT_ROOT (create directory if it doesn't exist).
#   2. Save `df_peak_results_national` as CSV with filename including MP number and date.
#   3. Filter to Allegheny County (FIPS 42003) → save as separate CSV.
#   4. Print file paths and row counts for both exports.
#   5. Print a summary table: top 10 counties by peak load delta (100% adoption scenario).
#
```

```
# INPUTS:
#   - `df_peak_results_national` (pd.DataFrame): from Step 9
#   - `primary_mp` (int): for filename
#   - `PROJECT_ROOT` (str): from config
# OUTPUTS: Two CSV files written to disk. Printed confirmation.
# CONSTRAINTS: Use pathlib.Path throughout. Include ISO date in filename.
#              Do not overwrite existing files - append a suffix if file exists.

# --- PLACEHOLDER: Opus/Copilot drafts implementation below ---
```

[ ]:

Handoff Notes for Claude Opus

What is complete (do not modify): - Steps 0–0c: TARE data loading, MP selection — working, tested - Step 1: EUSS annual CSV loading + 3-stage filter — working, tested

What needs implementation (in order): 1. Step 1 (NEW): BuildStockQuery install + AWS credential check 2. Step 2: OEDI Athena schema discovery — **must resolve column names before any other step** 3. Step 3: County geography mapping (FIPS/GISJOIN → shapefile) 4. Step 4: Adopter ID extraction from TARE results 5. Steps 5–6: Single-county test queries (Allegheny County first, always) 6. Step 7: Scenario demand profile + peak calculation 7. Step 8: Validation against EUSS built-in peak values 8. Step 9: National loop (only after Step 8 passes) 9. Step 10: Export

Critical constraint: ResStock 2022.1.1 (EUSS) only. Do not reference 2025.1.

Primary test case: Allegheny County, PA — FIPS 42003.

Code standards: Type hints on all functions. Google/NumPy docstrings. Fail fast.

If Athena table names are unknown: Check the OEDI S3 browser first: `https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2Fend-us` Then check the BuildStockQuery wiki for the correct initialization parameters.

[ ]:

Geospatial Visualization

[33]:
```
gdf_conus = None
gdf_alaska = None

try:
    gdf_states_raw = gpd.read_file(SHAPEFILE_PATH)
    _, gdf_conus, gdf_alaska = prepare_state_geodataframe(gdf_states_raw,␣
 ↪df_spark, merge_col='state')
    print(f"  Geodataframe prepared: CONUS={len(gdf_conus)},␣
 ↪AK={len(gdf_alaska)}")
```

```
except Exception as e:
    print(f"  Shapefile not loaded: {e} - skipping maps")
```

```
Shapefile not loaded: name 'df_spark' is not defined - skipping maps
```

[34]:
```
# Demand change map (diverging)
if gdf_conus is not None and gdf_alaska is not None:
    _, gdf_demand_conus, gdf_demand_alaska = prepare_state_geodataframe(
        gdf_states_raw, df_demand_state, merge_col='state'
    )
    create_choropleth_map(
        gdf_demand_conus, gdf_demand_alaska,
        column='elec_change_gwh',
        title='Electricity Demand Change Under 100% HP Adoption by State␣
 ↪(2022)',
        cbar_label='Electricity Demand Change (GWh)\n(positive = more grid␣
 ↪electricity needed)',
        output_path=os.path.join(PROJECT_ROOT,␣
 ↪"state_elec_demand_change_map_2022.png"),
        cmap='coolwarm', show_plot=True,
    )
    print("  Demand map generated")
else:
    print("  Maps skipped")
```

```
Maps skipped
```

Display Results

[35]:
```
# ============================================================================
# DISPLAY: DEMAND CHANGE
# ============================================================================

# print(f"\n===== DEMAND CHANGE (MP{primary_mp}, GWh, all fuels, 100% adoption)␣
 ↪=====\n")
# display(df_demand_state[['state', 'home_count', 'elec_change_gwh',
#                          'pct_elec_demand_change', 'site_energy_change_gwh',
#                          'pct_site_energy_change']])

# print(f"\n  DISPLAY COMPLETE")
```